ASP.NET Core supports the storage of application settings as environment variables for production applications and includes a feature called User Secrets for the safe storage of secret application settings during production. We will look at both of these options in the final chapter when we explore configuration.

13.4 Injection attacks

For a web application to be interactive, it must accept and process user input. Users can take advantage of this fact by injecting input, which, when processed under certain conditions, can affect the behavior of the application in ways the developers did not intend. In this section, we will take a detailed look at the two most common injection attacks: SQL injection and cross-site scripting. The side effects of both of these types of vulnerabilities can be catastrophic for users individually and your web application as a whole, so it is very important to understand how they arise and how you can protect yourself against them.

SQL injection attacks target your data access layer, while XSS attacks take advantage of your need to generate HTML dynamically. Given the potentially serious nature of the consequences that can arise from injection attacks, it should come as no surprise that the default behavior of well-designed frameworks is to protect you against them. In the case of SQL injection, EF Core provides great protection, as you will learn, and the Razor view engine mitigates XSS injection by default. There will be occasions in which the safety net provided by these tools is not available to you, so we will look at how to protect your application when that happens.

13.4.1 SQL injection

SQL injection is a vulnerability that allows an attacker to alter an SQL command the application executes, changing the expected behavior of the application. This can result in sensitive data exposure, data theft, data destruction, and even a compromised server. SQL injection vulnerabilities are introduced whenever you take user input and incorporate it into an SQL statement that is executed by the application. An attacker with knowledge of SQL can include text that alters the way the SQL is executed and the results it returns.

So far you haven’t written any SQL statements directly, but it is not unusual to do so in a real-world application, even when using an ORM like Entity Framework. You may need to write your own SQL statements when the SQL generated by the ORM is not suitable from